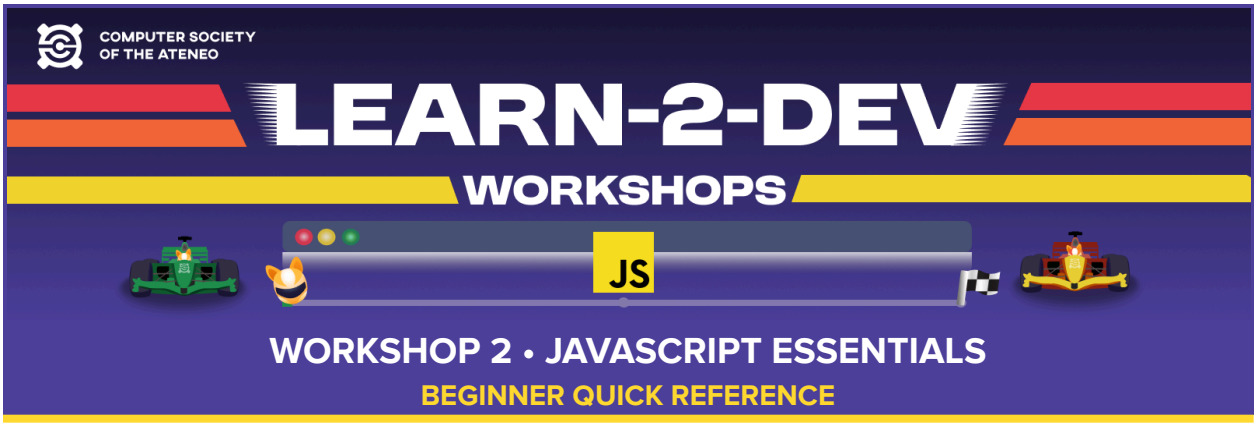


JavaScript: Values and Behavior

JavaScript adds behavior to the HTML. Save the code in a source file called script.js and link it inside the HTML `<head>` block:

HTML

```
<script src="script.js" defer></script>
```

`defer` runs the script after the HTML is read. Use **Developer Tools → Console** to check `console.log()` output. The examples on pages 1–3 are separate snippets.

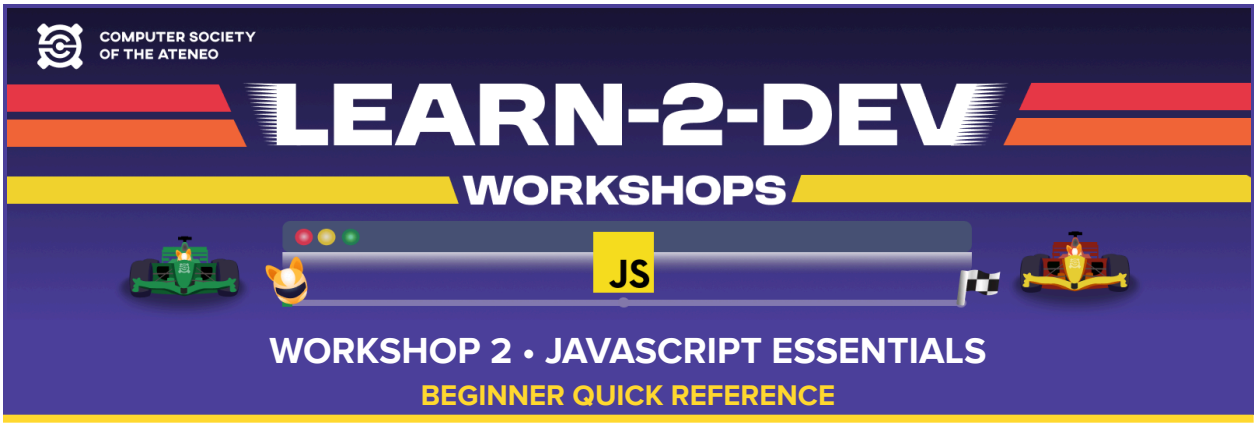
let, const, and Block Scope

It is good practice to use `const` to declare variables by default. Use `let` only when the value of that variable is expected to change. Both are limited to the `{ }` block where they are declared. They are preferred over the older `var`.

JavaScript

```
let clicks = 0;
clicks += 1;
const user = { name: "Alex" };
user = { id: 1234 }; // user.id → not allowed
user.name = "Sam"; // allowed
```

`const` prevents a variable from being reassigned; it does not freeze an object or array.



Type Conversion and Strict Equality

JavaScript may convert types automatically. `+` performs concatenation when strings are involved. Use `Number()` when numeric addition is desired. It is recommended to use `===` and `!==` over `==` and `!=` in enforcing strict equality. The latter may convert types before performing comparisons while the former does not.

```JavaScript
console.log("5" + 1);          // "51"
console.log(Number("5") + 1); // 6
console.log(5 == "5");         // true
console.log(5 === "5");        // false
```

Truthy, Falsy, and Missing Values

Truthy and falsy describe how JavaScript treats a value when checking a condition, such as in an `if` statement. Truthy values are treated as `true`, so the `if` block runs. Falsy values, treated as `false`, consequently skips the `if` block. `undefined` means no value was assigned; `null` is an intentional “no value.” `NaN`, short for Not A Number, means a numeric operation did not produce a valid number. In a condition, `false`, `0`, `""`, `null`, `undefined`, and `NaN` are falsy. Empty `arrays []` and `objects {}`, despite being empty, are truthy.

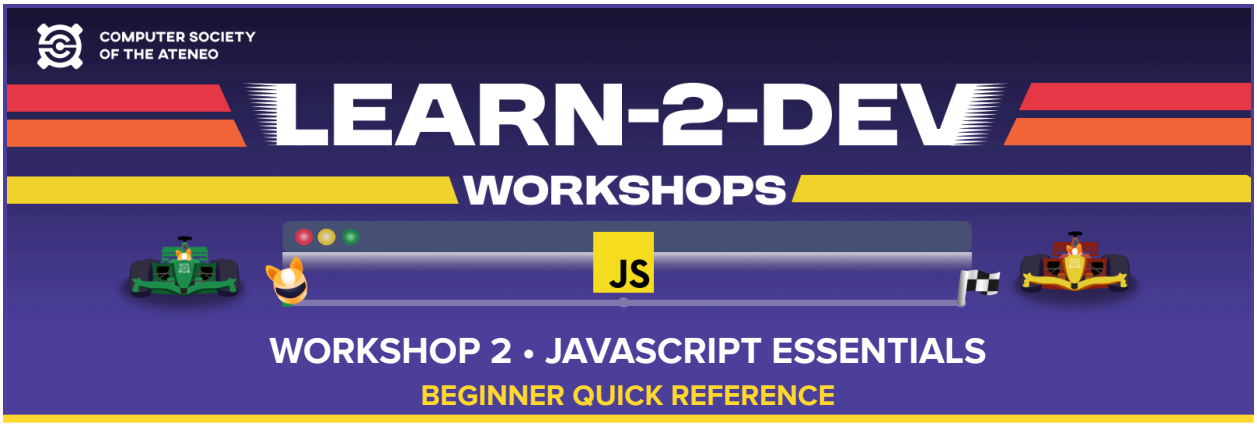
Objects, Arrays, and Callbacks

Objects and Template Literals

An `object` groups named properties. Use a *dot* (`.`) to read a property. Template literals are strings wrapped in backticks (). Use the placeholder `${variable}` to insert values into template literals.

```JavaScript
const user = { name: "Alex", score: 8 };
console.log(user.name); /* Alex */
console.log(`Hi, ${user.name}!`); // Hi, Alex!
```

Keep learning: MDN JavaScript beginner guide (developer.mozilla.org)



Arrow Functions and Callbacks

Arrow functions use `=>` without a function name in their syntax. Store one in a variable, or pass it directly as an anonymous callback—a function passed to another function for it to call.

Here, `n` is an input parameter. `n =>` and `(n) =>` both accept one input. Use `() =>` for none; multiple parameters, such as `(func, num) =>`, need parentheses.

An expressive body returns its value automatically. A `{ }` body can contain multiple statements across several lines; use `return` to send back a value:

```javascript
const timesTwo = n => n * 2;
const callback = (func, num) => {
  return func(num);
};

console.log(timesTwo(3));             // 6
console.log(callback(timesTwo, 4));   // 8
```

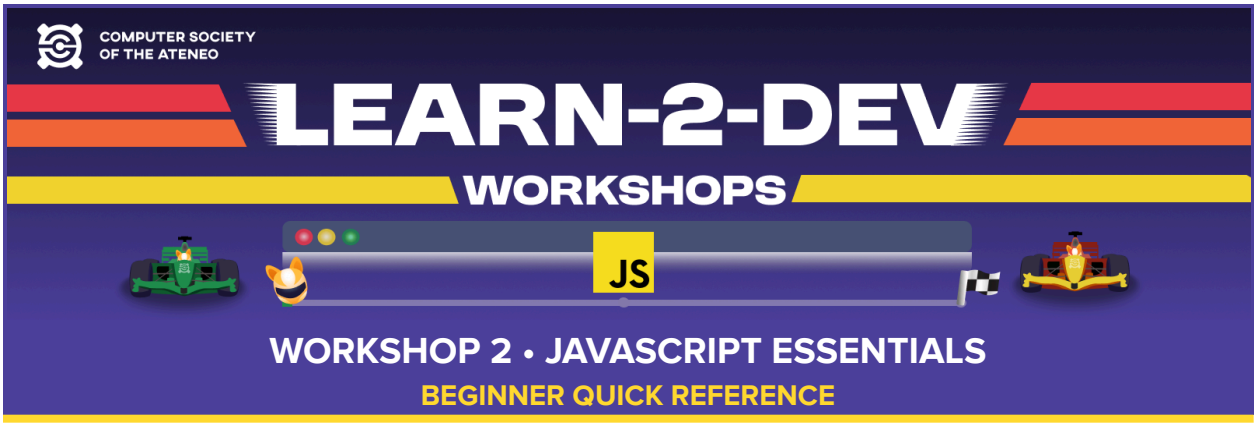
Array Helpers: map, filter, and forEach

Both `map()` and `filter()` take a callback function as an argument. `map()` returns a new array of the callback's results. `filter()` returns a new array of items from the original array that satisfy the condition in its callback.

```javascript
const scores = [4, 8, 10];
console.log(scores.map(n => n * 2)); // [8, 16, 20]
console.log(scores.filter(n => n >= 5)); // [8, 10]
scores.forEach(n => console.log(n)); // 4, then 8, then 10
```

`forEach()` performs an action for each item; it does not return a new array. `map()` and `filter()`, by themselves, do not change the original array.

[Keep learning: MDN JavaScript beginner guide (developer.mozilla.org)]



Objects and Arrays Use References

Reassigning an object or array to another variable does not produce a copy. Instead, both variables will share a reference to the same object or array. Changes to its contents are visible through either variable; reassigning a variable does not change the other reference.

JavaScript

```
const original = { score: 1 };
const shared = original;
shared.score = 2;
console.log(original.score); // 2

// Spread: unpack items or copy properties
const values = [2, 4]
console.log([...values, 6]); // [2, 4, 6]
console.log(...values);      // 2 4
const copy = {...original}   // shallow copy
```

Spread syntax (...), often called “unpacking,” expands array elements into a new array or a function call. In an object it copies properties: `const copy = { ...original }` makes a shallow copy, so nested objects remain shared.

JavaScript in the Browser

The Document Object Model, or DOM, is the browser’s representation of the HTML. `querySelector()` finds the first element matching a CSS selector; `querySelector(“#name”)` selects `id="name"`.

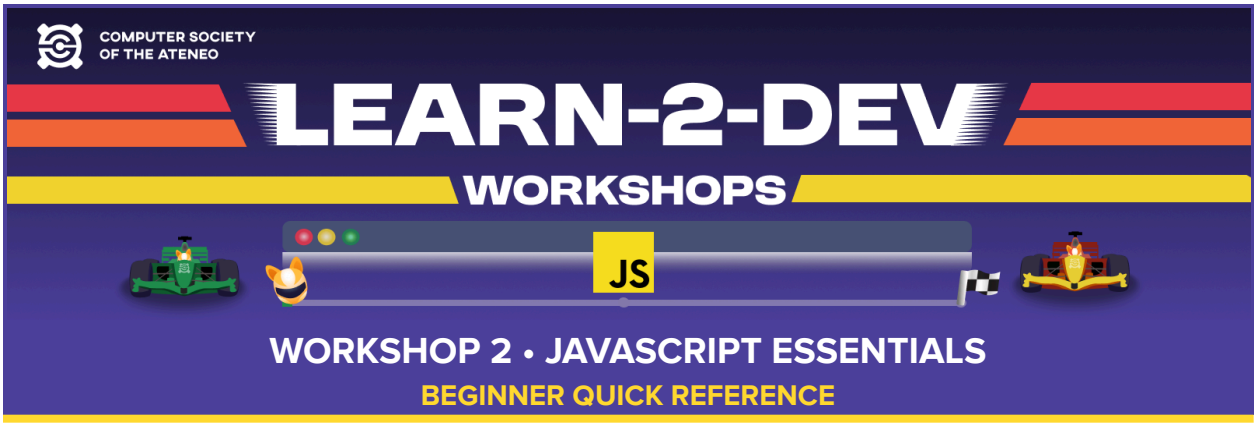
Example: Greet a Visitor

Add this HTML to the page and link script.js using `defer`, as shown on page 1.

HTML

```
<form id="greet">
  <input id="name" aria-label="Your name">
  <button>Say hello</button>
</form>
<p id="message">Enter your name above.</p>
```

Keep learning: MDN JavaScript beginner guide (developer.mozilla.org)



In script.js, use a callback to respond when the form is submitted:

```javascript
const form = document.querySelector("#greet");
const input = document.querySelector("#name");
const message = document.querySelector("#message");
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const name = input.value.trim() || "friend";
  message.textContent = `Hello, ${name}!`;
});
```

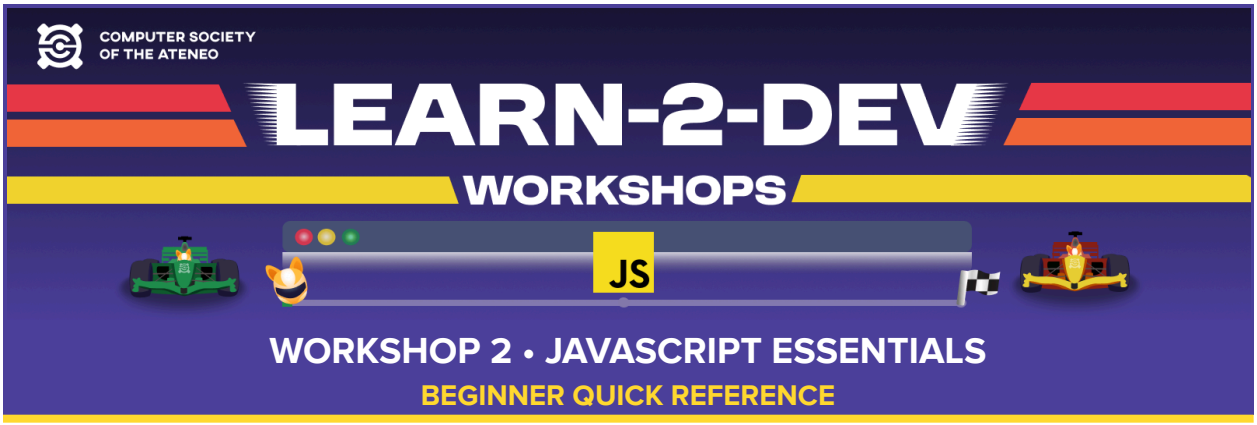
- `addEventListener()` registers the callback. The browser calls it when a `submit` event happens and supplies the event object.
- `preventDefault()` stops the normal form submission.
- `input.value` reads the input as a string; `.trim()` removes spaces from both ends.
- `||` uses "friend" when the trimmed input is empty.
- `message.textContent` changes what the <p> tag shows, displaying the greeting.

Change CSS Classes

```javascript
message.classList.toggle("highlight");
```

`toggle()` adds the class to the element if it is absent and removes it if present. Define the `.highlight` class in CSS to style the message.

Keep learning: MDN JavaScript beginner guide (developer.mozilla.org)



JavaScript Gotchas to Remember

- `querySelector()` returns null if nothing matches. Check IDs and script timing before using the selected element.
- Input values are strings, even for `type="number"`. Use `Number()` to convert its value before calculating; `Number.isNaN(value)` checks whether a value is Not A Number (`NaN`).
- A handler function can be passed to `addEventListener()` which the browser can call when the event occurs. Ensure that the function itself is passed by writing the function name without parentheses. Writing `handler()`, instead of simply `handler`, will call the function immediately and pass its return value—usually `undefined`—instead of saving the function for later.

```javascript
const handler = () => console.log("Submitted!");
form.addEventListener("submit", handler); // runs on submit
// Do not include the parenthesis
// Wrong: form.addEventListener("submit", handler());
```

Keep learning: MDN JavaScript beginner guide (developer.mozilla.org)